

Git Branches, JUnit Basics; CI-Pipeline

Git-Spiel

Machen Sie nun verschiedene Experimente mit Branches in Git, und starten Sie dabei jeweils mit einem frischen Klon der Vorgaben.

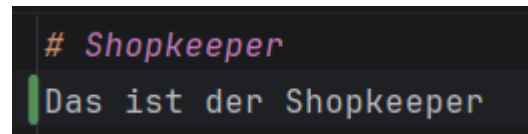
- Ändern Sie eine Datei, die im Branch end nicht verändert wurde. Erzeugen Sie mit diesen Änderungen auf dem master einen neuen Commit. Mergen Sie danach den Branch end in den master-Branch.

- `git checkout end`
- `git diff master --name-only`

Was verändert wurde:

`questlog.md`
`rucksack.md`
`stats.md`

- `git checkout master`
- Änderung der Datei `shopkeeper.md`:



```
# Shopkeeper
Das ist der Shopkeeper
```

- `git add shopkeeper.md`
- `git commit -m "Shopkeeper Text erweitert"`
- `git merge end`

Es kommt das raus:

`Merge branch 'end'`

`# Please enter a commit message to explain why this merge is necessary, #`
`especially if it merges an updated upstream into a topic branch.`

`#`

`# Lines starting with '#' will be ignored, and an empty message aborts`
`# the commit.`

- Man muss dann `:wq` machen, damit man den Merge beendet.
(`:w` - write (speichern) und `:q` – quit(beenden))

- Ändern Sie nun eine Datei, die auch im Branch end verändert wurde. Achten Sie dabei darauf, die Änderung an einer anderen Stelle in der Datei vorzunehmen. Erzeugen Sie mit diesen Änderungen auf dem master einen neuen Commit. Mergen Sie danach den Branch end in den master-Branch.

- Änderung der Datei stats.md:

Property	Value
health	8
experience	40
hunger	0
weapon	sword (3 dmg)
armor	light (2 dmg)
thirst	1

- `git add stats.md`
 - `git commit -m "thirst added"`
 - `git merge end`
- Wie (2), aber ändern Sie nun eine Stelle, die auch im Branch end verändert wurde. Erzeugen Sie mit diesen Änderungen auf dem master einen neuen Commit. Mergen Sie danach den Branch end in den master-Branch. Was passiert, wenn die Änderung im master identisch zu der in end ist? Was passiert, wenn die Änderung im master anders ist als in end?

- Änderung der Datei rucksack.md:

Slot	Content
0	1 Heiltrank
1	1 Stein
2	
3	
4	
5	
6	

- `git add rucksack.md`
- `git commit -m "Stein added"`
- `git merge end`

Es kommt folgende Fehlermeldung:

Auto-merging rucksack.md

CONFLICT (content): Merge conflict in rucksack.md

Automatic merge failed; fix conflicts and then commit the result.

- Da Stein und Amulett an gleicher Stelle ist, entsteht ein Konflikt. Diesen muss man manuell lösen und dann commiten.

----> `[master c18d56d] Merge branch 'end'`

- Bei gleicher Änderung entsteht kein Konflikt, da beide Änderungen identisch sind.

- Wie (2), aber setzen Sie bitte den Branch end auf die Spitze von master, bevor Sie end in master mergen.

- Änderung der Datei stats.md:

Property	Value
health	8
experience	40
hunger	0
weapon	sword (3 dmg)
armor	light (2 dmg)
thirst	1

- `git add stats.md`
- `git commit -m "thirst added"`
- `git checkout end`
- `git merge master`
- `git checkout master`
- `git merge end`
- Beim merge von end in master entsteht kein neuer Commit. Im log ist nur der merge von master in end zusehen:

```
commit 908f8c47f4adca3ff0ff6565dcd36b71746268a6 (HEAD -> master, end)
Merge: 6405bd9 5c88b0c
Author: Nick Resner <resner2007@gmail.com>
Date: Thu May 7 16:47:20 2026 +0200

Merge branch 'master' into end
```

- Es entsteht ein Fast-Forward-Merge
(Quelle: <https://stackoverflow.com/questions/29673869/what-is-fast-forwarding>)

Katzen-Café

Gradle

Erstellen Sie eine Gradle-Konfiguration für das Java-Projekt und fügen Sie als externe Abhängigkeiten JUnit (Version 6.x) sowie das Gradle-Plugin Spotless hinzu. Passen Sie die Konfiguration so an, dass Sie:

- `main()` in `catcafe.Main` über Gradle ausführen können, und
- JUnit-Tests mit Gradle ausführen können, und
- den Code mit `google-java-format` formatieren können. Können Sie für den Google-Java-Formatter die Einrückung auf 4 Leerzeichen (statt der Default 2 Leerzeichen) anpassen? Lange Strings sollen ebenfalls umgebrochen werden (falls nötig).

- Können Sie für den Google-Java-Formatter die Einrückung auf 4 Leerzeichen (statt der Default 2 Leerzeichen) anpassen? ----- Nein XD

build.gradle Datei:

```
plugins {  
    id 'java'  
    id("com.diffplug.spotless") version "8.4.0"  
    id 'application'  
}  
  
repositories{  
    mavenCentral()  
}  
  
dependencies {  
    testImplementation 'org.junit.jupiter:junit-jupiter:6.0.0'  
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher:1.10.0'  
}  
  
test {  
    useJUnitPlatform()  
}  
  
application {  
    mainClass = 'catcafe.Main'  
}  
  
spotless {  
    java{googleJavaFormat()  
        //Entfernt Leerzeichen am Zeilenende  
        trimTrailingWhitespace()  
        //Jede Datei endet mit nur einer leeren Zeile  
        endWithNewline()  
    }  
}
```

JUnit

Erstellen Sie mit JUnit mindestens 10 unterschiedliche Testfälle für die Klasse CatCafe. Achten Sie auf den Aufbau der Testfälle und nutzen Sie das Muster "given - when - then". Achten Sie auch auf passende Methodennamen.

Ein JUnit-Test besteht immer aus:

Testklasse → liegt unter src/test/java

Testmethoden → mit @Test

Assertions → prüfen, ob das Ergebnis stimmt

„The Given-When-Then rule is a common way to structure tests, and it is used to make tests more readable and maintainable. The basic structure of the rule is as follows:

Given: this section sets up the initial state of the system, including any required dependencies or data.

When: this section triggers the action or behavior that we want to test.

Then: this section asserts that the expected outcome has occurred.“

(Quelle: <https://medium.com/@gitaeklee/given-when-then-junit-test-ba49564303e7>)

Assertions, die ich verwende:

- **assertEquals(expected, actual):**
→ Prüft, ob zwei Werte gleich sind. Es gibt Varianten für primitive Typen und Objekte.
- **assertNull(actual) / assertNotNull(actual):**
→ Stellt sicher, dass ein Objekt null ist oder eben nicht null ist.
- **import static org.junit.jupiter.api.Assertions.*;**

(Quelle: <https://docs.junit.org/5.0.1/api/org/junit/jupiter/api/Assertions.html>)

Dinge, die man bei den Methoden aus der Klasse CatCafe testen kann:

```
public void addCat(FelineOverLord cat) {...}
```

→ Wurde die Katze wirklich dem CatCafe hinzugefügt?

```
public long getCatCount(){...}
```

→ Wird die richtige Anzahl ausgegeben?

➔ Wenn es keine Katzen gibt, kommt 0 raus?

```
public FelineOverLord getCatByName(String name) {...}
```

➔ Wird die richtige Katze ausgegeben?

➔ Wenn keine Katze mit dem Namen existiert, kommt NULL raus?

➔ Kommt NULL raus, wenn man als Namen NULL sucht?

```
public FelineOverLord getCatByWeight(int minWeight, int maxWeight) {...}
```

➔ Wird die richtige Katze ausgegeben?

➔ Wenn die Katze nicht im Bereich liegt, kommt NULL raus?

➔ Kommt NULL raus, wenn man minWeight und maxWeight vertauscht?

➔ Kommt NULL raus, wenn das minWeight unter 0 ist?

```
String accept(TreeVisitor<FelineOverLord> visitor) {...}
```

➔ ???

Test Klasse:

```
package catcafe;
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
class CatCafeTest {

    @Test
    void CatCountIncreases() {
        // given
        CatCafe cafe = new CatCafe();
        FelineOverLord cat = new FelineOverLord("Castorice", 3);

        // when
        cafe.addCat(cat);

        // then
        assertEquals(1, cafe.getCatCount());
    }

    @Test
    void CatCorrectCountReturned() {
        // given
        CatCafe cafe = new CatCafe();
        FelineOverLord cyrene = new FelineOverLord("Cyrene", 2);
        FelineOverLord hyacinthia = new FelineOverLord("Hyacinthia", 3);
        cafe.addCat(cyrene);
        cafe.addCat(hyacinthia);

        // when
        long count = cafe.getCatCount();

        // then
        assertEquals(2, count);
    }
}
```

```

@Test
void ZeroCatsReturned() {
    // given
    CatCafe cafe = new CatCafe();

    // when
    long count = cafe.getCatCount();

    // then
    assertEquals(0, count);
}

```

```

@Test
void CorrectCatByNameReturned() {
    // given
    CatCafe cafe = new CatCafe();
    FelineOverLord aglaea = new FelineOverLord("Aglaea", 3);
    cafe.addCat(aglaea);
    FelineOverLord evernight = new FelineOverLord("Evernight", 4);
    cafe.addCat(evernight);
    FelineOverLord phainon = new FelineOverLord("Phainon", 5);
    cafe.addCat(phainon);

    // when
    FelineOverLord result = cafe.getCatByName("Evernight");

    // then
    assertEquals(evernight, result);
}

```

```

@Test
void NullByWrongNameReturned() {
    // given
    CatCafe cafe = new CatCafe();
    FelineOverLord mydei = new FelineOverLord("Mydei", 5);
    cafe.addCat(mydei);

    // when
    FelineOverLord result = cafe.getCatByName("Phainon");

    // then
    assertNull(result);
}

```

```

@Test
void NullByNullNameReturned() {
    // given
    CatCafe cafe = new CatCafe();
    FelineOverLord hyselins = new FelineOverLord(„Hyselins", 8);
    cafe.addCat(hyselins);

    // when

```

```

    FelineOverLord result = cafe.getCatByName(null);

    // then
    assertNull(result);
}

@Test
void CorrectCatByWeightReturned() {
    // given
    CatCafe cafe = new CatCafe();
    FelineOverLord tribbie = new FelineOverLord("Tribbie", 5);
    cafe.addCat(tribbie);
    FelineOverLord cypher = new FelineOverLord("Cypher", 2);
    cafe.addCat(cypher);

    // when
    FelineOverLord result = cafe.getCatByWeight(5, 6);

    // then
    assertEquals(tribbie, result);
}

@Test
void NullByIncorrectWeightReturned() {
    // given
    CatCafe cafe = new CatCafe();

    // when
    FelineOverLord result = cafe.getCatByWeight(10, 5);

    // then
    assertNull(result);
}

@Test
void NullByNegativeWeightReturned() {
    // given
    CatCafe cafe = new CatCafe();

    // when
    FelineOverLord result = cafe.getCatByWeight(-1, 5);

    // then
    assertNull(result);
}

@Test
void NullByWrongWeightReturned() {
    // given
    CatCafe cafe = new CatCafe();
    FelineOverLord anaxa = new FelineOverLord("Anaxa", 4);
    FelineOverLord cerydra = new FelineOverLord("Cerydra", 6);
    cafe.addCat(anaxa);
    cafe.addCat(cerydra);
    // when
    FelineOverLord result = cafe.getCatByWeight(15, 20);

```



```

    // then
    assertNull(result);
}
}

```

Frage die beim Testen aufgekomen ist:

```

47
48     @Test new *
49     void CorrectCatByNameReturned() {
50         // given
51         CatCafe cafe = new CatCafe();
52         FelineOverLord aglaea = new FelineOverLord( name: "Aglaea", weight: 4);
53         cafe.addCat(aglaea);
54         FelineOverLord evernight = new FelineOverLord( name: "Evernight", weight: 4);
55         cafe.addCat(evernight);
56         FelineOverLord phainon = new FelineOverLord( name: "Phainon", weight: 4);
57         cafe.addCat(phainon);
58
59         long count = cafe.getCatCount();
60
61         // then
62         assertEquals( expected: 3, count);
63     }
64
65
66
67     @Test new *
68     void NullByWrongNameReturned() {

```

returned x

1 test failed 1 test total, 31 ms

```

> Task :compileTestJava
> Task :processTestResources NO-SOURCE
> Task :testClasses

Expected :3
Actual    :1
<Click to see difference>

```

Beim gleichem Gewicht zählt er nur eine Katze... warum???

Warum sind die von Ihnen formulierten Testfälle relevant?

Testet 4 von 5 Methoden auf verschiedene Fälle. Die eine Methode habe ich leider nicht verstanden.... Gewährt die Richtigkeit der Ausgaben der Methoden bei verschiedenen Parametern und somit relevant.

Warum halten Sie die formulierten Testfälle für unterschiedlich?

Sie testen unterschiedliche Fälle und unterschiedliche Methoden. Zum Beispiel was passiert wenn Gewicht vertauscht, negativ oder richtig ist.

Remotes und CI-Pipeline

(Quelle: https://youtu.be/R8_veQiYBjI?si=9Rnnz-aWktRZN9rC; Andere Videos/Websites)

Wichtige Notizen für mich und erklärungen:

runs-on: ubuntu-latest

→ GitHub startet eine frische Linux-VM.

uses: actions/checkout@v4

→ Holt deinen Code aus GitHub in die VM.

Gradle kann normalerweise im Hintergrund einen Daemon-Prozess laufen lassen, der Builds beschleunigt. In CI-Systemen wie GitHub Actions ist das aber nicht sinnvoll, weil:

→ jede Pipeline in einer frischen VM läuft

→ der Daemon nicht wiederverwendet wird

→ der Daemon manchmal nicht sauber beendet wird

→ der Daemon mehr RAM verbraucht

→ Builds dadurch instabil werden können

run: chmod +x gradlew

→ Macht gradlew ausführbar. Wichtig, weil GitHub Actions Dateien ohne Ausführungsrechte klonet.